

What is agentic engineering?

I use the term **agentic engineering** to describe the practice of developing software with the assistance of coding agents.

What are **coding agents**? They're agents that can both write and execute code. Popular examples include [Claude Code](#), [OpenAI Codex](#), and [Gemini CLI](#).

What's an **agent**? Clearly defining that term is a challenge that has frustrated AI researchers since [at least the 1990s](#) but the definition I've come to accept, at least in the field of Large Language Models (LLMs) like GPT-5 and Gemini and Claude, is this one:

Agents run tools in a loop to achieve a goal

The "agent" is software that calls an LLM with your prompt and passes it a set of tool definitions, then calls any tools that the LLM requests and feeds the results back into the LLM.

For coding agents, those tools include one that can execute code.

You prompt the coding agent to define a goal. The agent then generates and executes code in a loop until that goal has been met.

Code execution is the defining capability that makes agentic engineering possible. Without the ability to directly run the code, anything output by an LLM is of limited value. With code execution, these agents can start iterating towards software that demonstrably works.

Agentic engineering

Now that we have software that can write working code, what is there left for us humans to do?

The answer is *so much stuff*.

Writing code has never been the sole activity of a software engineer. The craft has always been figuring out *what* code to write. Any given software problem has dozens of potential solutions, each with their own tradeoffs. Our job is to navigate those options and find the ones that are the best fit for our unique set of circumstances and requirements.

Getting great results out of coding agents is a deep subject in its own right, especially now as the field continues to evolve at a bewildering rate.

We need to provide our coding agents with the tools they need to solve our problems, specify those problems in the right level of detail, and verify and iterate on the results until we are confident they address our problems in a robust and credible way.

LLMs don't learn from their past mistakes, but coding agents can, provided we deliberately update our instructions and tool harnesses to account for what we learn along the way.

Used effectively, coding agents can help us be much more ambitious with the projects we take on. Agentic engineering should help us produce more, better quality code that solves more impactful problems.

Isn't this just vibe coding?

The term "vibe coding" was [coined by Andrej Karpathy](#) in February 2025 - coincidentally just three weeks prior to the original release of Claude Code - to describe prompting LLMs to write code while you "forget that the code even exists".

Some people extend that definition to cover any time an LLM is used to produce code at all, but I think that's a mistake. Vibe coding is more useful in its original definition - we need a term to describe unreviewed, prototype-quality LLM-generated code that distinguishes it from code that the author has brought up to a production ready standard.

About this guide

Just like the field it attempts to cover, *Agentic Engineering Patterns* is very much a work in progress. My goal is to identify and describe patterns for working with these tools that demonstrably get results, and that are unlikely to become outdated as the tools advance.

I'll continue adding more chapters as new techniques emerge. No chapter should be considered finished. I'll be updating existing chapters as our understanding of these patterns evolves.